EX No:10

Date:

Apply Regularization & Dimension Reduction Methods using Auto Data

10. We will now consider the Boston housing data set, from the ISLP library.

LOAD LIBRARY:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from ISLP import load_data
from matplotlib.pyplot import subplot
```

CODE:

```
Boston = load_data('Boston')
Boston.info()
```

OUTPUT:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 13 columns):
 #   Column   Non-Null Count  Dtype  
---  ------   --------------  -----  
 0   crim     506 non-null    float64
 1   zn       506 non-null    float64
 2   indus    506 non-null    float64
 3   chas     506 non-null    int64  
 4   nox      506 non-null    float64
 5   rm       506 non-null    float64
 6   age      506 non-null    float64
 7   dis      506 non-null    float64
 8   rad      506 non-null    int64  
 9   tax      506 non-null    int64  
 10  ptratio  506 non-null    float64
 11  lstat    506 non-null    float64
 12  medv     506 non-null    float64
dtypes: float64(10), int64(3)
memory usage: 51.5 KB
```

(a) Based on this data set, provide an estimate for the population mean of medv. Call this estimate $\hat{\mu}$.

CODE:

```
mu_hat = Boston['medv'].mean()
mu_hat
```

OUTPUT:

```
np.float64(22.532806324110677)
```

(b) Provide an estimate of the standard error of $\hat{\mu}$. Interpret this result. Hint: We can compute the standard error of the sample mean by dividing the sample standard deviation by the square root of the number of observations.

CODE:

```
SE_mu_hat = Boston['medv'].std()/(Boston.shape[0])**0.5
SE_mu_hat
```

OUTPUT:

```
np.float64(0.40886114749753505)
```

(c) Now estimate the standard error of $\hat{\mu}$ using the bootstrap. How does this compare to your answer from (b)?

CODE:

```
means = [ ]
for _ in range(1000):
   means.append(Boston['medv'].sample(n=Boston.shape[0],replace=True,
       random_state=None).mean())
boot_std = np.std(means)
boot_std
```

OUTPUT:

```
np.float64(0.4134095806706386)
```

(d) Based on your bootstrap estimate from (c), provide a 95% confidence interval for the mean of medv. Compare it to the results obtained by using Boston['medv'].std() and the two standard error rule (3.9).

CODE:

```
print('The 95% confidence interval for the mean of medv is', mu_hat - 2*boot_std, mu_hat + 2*boot_std)
```

OUTPUT:

```
The 95% confidence interval for the mean of medv is 21.7059871627694 23.359625485451954
```

(e) Based on this data set, provide an estimate, $\hat{\mu}_{med}$, for the median value of medv in the population.

CODE:

```
mu  med = Boston['medv'].median()
mu_med
```

OUTPUT:

```
np.float64(21.2)
```

(f) We now would like to estimate the standard error of $\hat{\mu}_{med}$. Unfortunately, there is no simple formula for computing the standard error of the median. Instead, estimate the standard error of the median using the bootstrap. Comment on your findings.

CODE:

```
medians = [ ]
for _ in range(10000):
    medians.append(Boston['medv'].sample(n=Boston.shape[0], replace=True).median())
boot_med_std = np.std(medians)
boot_med_std
```

OUTPUT:

```
np.float64(0.3789835204530663)
```

(g) Based on this data set, provide an estimate for the tenth percentile of medv in Boston census tracts. Call this quantity μ̂0.1. (You can use the np.percentile() function.)

CODE:

```
mu_01 = np.percentile(Boston['medv'], 10)
mu_01
```

OUTPUT:

```
np.float64(12.75)
```

(h) Use the bootstrap to estimate the standard error of μ̂0.1. Comment on your findings.

CODE:

```
percentiles = [ ]
for _ in range(5000):
    sample = Boston['medv'].sample(n=Boston.shape[0], replace=True)
    percentiles.append(np.percentile(sample, 10))
boot_mu01_std = np.std(percentiles)
boot_mu01_std
```

OUTPUT:

```
np.float64(0.5010697531282445)
```

CODE:

```
def boot_SE(func, D, n=None, B=1000, seed=0):
    rng = np.random.default_rng(seed)
    first_, second_ = 0, 0
    n = n or D.shape[0]
    for _ in range(B):
        idx = rng.choice(D.index, n, replace=True)
        value = func(D, idx)
        first_ += value
        second_ += value**2
    return np.sqrt(second_ / B - (first_ / B)**2)

def boot_fn(df, idx):
    return np.percentile(df.iloc[idx], 10)
```

```
boot_SE(boot_fn, Boston['medv'], B=10000, seed=1)
```

OUTPUT:

```
np.float64(0.5079360794215755)
```

INTERPRETATION:

- (a) The estimated population mean of medv is 22.53 (about $22,500, as medv is in $1000s).
- (b) The standard error of the mean from the formula s / sqrt(n) is 0.4089, so the sample mean typically differs from the true mean by about 0.41 (about $409).
- (c) The bootstrap standard error (1,000 resamples) is 0.4134, almost identical to the formula value of 0.4089, so the two methods agree.
- (d) The 95% confidence interval for the mean from the bootstrap (mean +/- 2 SE) is about 21.71 to 23.36. The usual two-standard-error rule using the formula SE gives practically the same interval.
- (e) The median of medv is 21.2, below the mean of 22.53, which indicates a right-skewed distribution in which expensive tracts pull the mean up.
- (f) The bootstrap standard error of the median (10,000 resamples) is 0.379, slightly smaller than for the mean (0.41), so the median is estimated about as precisely and is less affected by extreme values.
- (g) The tenth percentile of medv is 12.75: about 10% of tracts have a median home value of $12,750 or less.
- (h) The bootstrap standard error of the tenth percentile is about 0.501 (0.508 using the boot_SE function), larger than for the mean or median, because tail quantiles rest on fewer observations and are therefore estimated less precisely.